Java Inheritance — Detailed Notes

1. What is Inheritance?

Inheritance is an OOP concept in Java where one class **acquires the properties and behaviors of another class**.
In simple words:

> A child class can use the variables and methods of its parent class.

It helps us achieve **code reusability**.

Real-life example

Suppose we have:

- `Animal` → common properties and behaviors
- `Dog` → specific type of Animal
- `Cat` → specific type of Animal

Instead of writing `eat()` and `sleep()` again in `Dog` and `Cat`, we can put them in `Animal`.

```
class Animal {
  void eat() {
    System.out.println("Animal is eating");
  }

  void sleep() {
    System.out.println("Animal is sleeping");
  }
}

class Dog extends Animal {
  void bark() {
    System.out.println("Dog is barking");
  }
}
```

Now:

```
Dog d = new Dog();

d.eat();
d.sleep();
d.bark();
```

Output:

```
Animal is eating
Animal is sleeping
Dog is barking
```

`Dog` inherited `eat()` and `sleep()` from `Animal`.

2. Important Terminology

Consider:

```
class Animal {

}

class Dog extends Animal {

}
```

Here:

TermMeaning`Animal`Parent / Superclass / Base class`Dog`Child / Subclass / Derived class`extends`Keyword used for class inheritance`Dog extends Animal`Dog inherits from Animal

Relationship

```
   Animal
     ↑
     |
    Dog
```

We can say:

> Dog **IS-A** Animal.

This is called an **IS-A relationship**.

3. Syntax of Inheritance

```
class Child extends Parent {

}
```

Example:

```
class Vehicle {

  void start() {
    System.out.println("Vehicle starts");
  }
}

class Car extends Vehicle {

  void drive() {
    System.out.println("Car is driving");
  }
}
```

Usage:

```
Car c = new Car();

c.start();
c.drive();
```

The `Car` object can access the inherited `start()` method.

4. Why Do We Need Inheritance?

The major purpose of inheritance is **code reusability**.

Without inheritance:

```
class Dog {

  void eat() {
    System.out.println("Eating");
  }

  void sleep() {
    System.out.println("Sleeping");
  }
}
class Cat {

  void eat() {
    System.out.println("Eating");
  }
```

```
  void sleep() {
    System.out.println("Sleeping");
  }
}
```

The same code is repeated.

With inheritance:

```
class Animal {

  void eat() {
    System.out.println("Eating");
  }

  void sleep() {
    System.out.println("Sleeping");
  }
}
class Dog extends Animal {

  void bark() {
    System.out.println("Barking");
  }
}
class Cat extends Animal {

  void meow() {
    System.out.println("Meowing");
  }
}
```

Now common functionality exists only once.

5. What Can a Child Class Inherit?

A child class can inherit accessible members of the parent class.

For example:

```
class Parent {

  int age = 50;

  void show() {
    System.out.println("Parent");
```

```
  }
}

class Child extends Parent {

  void display() {
    System.out.println(age);
    show();
  }
}
```

`Child` can use:

```
age
show()
```

because they are accessible.

6. Types of Inheritance in Java

Java supports several inheritance structures.

1. Single Inheritance

```
A
|
B
```

Example:

```
class A {
  void showA() {
    System.out.println("A");
  }
}

class B extends A {
  void showB() {
    System.out.println("B");
  }
}
```

2. Multilevel Inheritance

```
A
```

```
|
B
|
C
```

Example:

```
class Animal {

  void eat() {
    System.out.println("Eating");
  }
}

class Dog extends Animal {

  void bark() {
    System.out.println("Barking");
  }
}

class Puppy extends Dog {

  void play() {
    System.out.println("Playing");
  }
}
```

Now:

```
Puppy p = new Puppy();

p.eat();
p.bark();
p.play();
```

`Puppy` gets functionality from both:

```
Animal
  ↓
 Dog
  ↓
Puppy
```

7. Hierarchical Inheritance

Multiple child classes inherit from the same parent.

```
    Animal
   /   \
  Dog   Cat
```

Example:

```
class Animal {

  void eat() {
    System.out.println("Eating");
  }
}

class Dog extends Animal {

  void bark() {
    System.out.println("Barking");
  }
}

class Cat extends Animal {

  void meow() {
    System.out.println("Meowing");
  }
}
```

Both `Dog` and `Cat` can use:

```
eat();
```

But `Dog` cannot directly use:

```
meow();
```

because `meow()` belongs to `Cat`.

8. Multiple Inheritance

Multiple inheritance means one class inherits from **multiple classes**.

Conceptually:

```
  A   B
   \  /
    C
```

For example:

```
class A {
  void show() {
    System.out.println("A");
  }
}

class B {
  void show() {
    System.out.println("B");
  }
}
```

Java does **not** allow:

```
class C extends A, B {

}
```

This produces a compilation error.

Why?

Because of the **Diamond Problem**.

Suppose both `A` and `B` have:

```
show()
```

Then Java would have difficulty determining which `show()` should be inherited by `C`.

Therefore:

> Java does not support multiple inheritance through classes.

9. Multiple Inheritance Using Interfaces

Java can achieve multiple inheritance through **interfaces**.

```
interface A {

  void show();
}

interface B {
```

```
  void display();
}

class C implements A, B {

  public void show() {
    System.out.println("Show");
  }

  public void display() {
    System.out.println("Display");
  }
}
```

Here:

```
class C implements A, B
```

is valid.

So remember:

> Java does not support multiple inheritance using classes, but it supports multiple inheritance using interfaces.

10. Hybrid Inheritance

Hybrid inheritance is a combination of two or more types of inheritance.

For example:

```
   A
  / \
 B  C
  \ /
   D
```

Java does not support such structures directly through classes because they can create ambiguity.

Interfaces can be used to achieve complex inheritance designs.

11. `extends` Keyword

The `extends` keyword is used when one class inherits another class.

```
class Parent {
```

```
}

class Child extends Parent {

}
```

A class can extend only **one class**.

Valid:

```
class B extends A {
}
```

Invalid:

```
class C extends A, B {
}
```

12. Inheritance and Constructors

A very important concept:

> Constructors are **not inherited**.

Example:

```
class Parent {

  Parent() {
    System.out.println("Parent Constructor");
  }
}

class Child extends Parent {

  Child() {
    System.out.println("Child Constructor");
  }
}
```

When we create:

```
Child c = new Child();
```

Output:

```
Parent Constructor
Child Constructor
```

Why?

Because before the child constructor executes, the parent constructor executes.

13. `super()` Keyword

`super()` is used to call the constructor of the parent class.

Example:

```
class Parent {

  Parent() {
    System.out.println("Parent");
  }
}

class Child extends Parent {

  Child() {
    super();
    System.out.println("Child");
  }
}
```

Output:

```
Parent
Child
```

Important point

If you don't explicitly write:

```
super();
```

Java automatically inserts it as the first statement of the child constructor, provided the parent has an accessible no-argument constructor.

So:

```
Child() {
  System.out.println("Child");
}
```

is effectively:

```
Child() {
```

```
  super();
  System.out.println("Child");
}
```

14. `super` Keyword

`super` refers to the **immediate parent class object/member context**.
It can be used for:

1. Calling parent variable

```
class Parent {

  int x = 10;
}

class Child extends Parent {

  int x = 20;

  void show() {
    System.out.println(x);
    System.out.println(super.x);
  }
}
```

Output:

```
20
10
```

2. Calling parent method

```
class Parent {

  void show() {
    System.out.println("Parent");
  }
}

class Child extends Parent {

  void show() {
    System.out.println("Child");
```

```
    super.show();
  }
}
```

Output:

```
Child
Parent
```

3. Calling parent constructor

```
class Parent {

  Parent() {
    System.out.println("Parent Constructor");
  }
}

class Child extends Parent {

  Child() {
    super();
    System.out.println("Child Constructor");
  }
}
```

15. Method Overriding

One of the most important concepts associated with inheritance is **method overriding**. When the child class provides its own implementation of a method already defined in the parent class, it is called **method overriding**.

Example:

```
class Animal {

  void sound() {
    System.out.println("Animal makes sound");
  }
}

class Dog extends Animal {

  @Override
  void sound() {
```

```
    System.out.println("Dog barks");
  }
}
```

Now:

```
Dog d = new Dog();

d.sound();
```

Output:

```
Dog barks
```

The child version is executed.

16. `@Override` Annotation

It is recommended to use:

```
@Override
```

Example:

```
class Dog extends Animal {

  @Override
  void sound() {
    System.out.println("Bark");
  }
}
```

It tells the compiler:

> "I intend to override a method from the parent class."

If you make a mistake in the method signature, the compiler can detect it.

17. Rules of Method Overriding

For overriding:

Rule 1: Same method name

Parent:

```
void show()
```

Child:

```
void show()
```

Rule 2: Same parameters

Parent:

```
void show(int x)
```

Child:

```
void show(int x)
```

Rule 3: Return type must be compatible

For normal overriding, the return type must be the same or a **covariant return type**.
Example:

```
class Animal {
  Animal getAnimal() {
    return this;
  }
}

class Dog extends Animal {
  @Override
  Dog getAnimal() {
    return this;
  }
}
```

This is valid because `Dog` is a subtype of `Animal`.

18. Access Modifiers and Overriding

A child cannot reduce the visibility of an overridden method.
For example:

```
class Parent {

  public void show() {

  }
}
```

This is invalid:

```
class Child extends Parent {

  @Override
  protected void show() {

  }
}
```

because:

```
public → protected
```

reduces visibility.

But this is allowed:

```
protected → public
```

because visibility is increased.

19. `private` Members and Inheritance

Private members belong to the parent class and cannot be directly accessed by the child.

Example:

```
class Parent {

  private int age = 50;
}

class Child extends Parent {

  void show() {
    System.out.println(age); // ERROR
  }
}
```

The child cannot directly access `age`.

However, the parent can expose it through methods:

```
class Parent {

  private int age = 50;
```

```
  public int getAge() {
    return age;
  }
}

class Child extends Parent {

  void show() {
    System.out.println(getAge());
  }
}
```

This is one reason **encapsulation** is important.

20. `protected` and Inheritance

`protected` members are especially useful in inheritance.

```
class Parent {

  protected int age = 50;
}

class Child extends Parent {

  void show() {
    System.out.println(age);
  }
}
```

The child can access the protected member.

21. `final` and Inheritance

The `final` keyword can prevent inheritance or overriding.

Final class

```
final class Animal {

}
```

You cannot do:

```
class Dog extends Animal {

}
```

because `Animal` is final.

Final method

```
class Animal {

  final void eat() {
    System.out.println("Eating");
  }
}
```

The child cannot override it:

```
class Dog extends Animal {

  void eat() { // ERROR
    System.out.println("Dog eating");
  }
}
```

22. Object Class and Inheritance

Every Java class ultimately inherits from:

```
Object
```

For example:

```
class Student {

}
```

is conceptually part of:

```
Object
 ↓
Student
```

`Object` provides important methods such as:

```
toString()
equals()
```

```
hashCode()
getClass()
```

Therefore:

```
Student s = new Student();

s.toString();
s.equals(...);
s.getClass();
```

are available because `Student` ultimately inherits from `Object`.

23. Inheritance and Polymorphism

Inheritance is closely connected with **runtime polymorphism**.

Example:

```
class Animal {

  void sound() {
    System.out.println("Animal sound");
  }
}

class Dog extends Animal {

  @Override
  void sound() {
    System.out.println("Bark");
  }
}
```

Now:

```
Animal a = new Dog();

a.sound();
```

Output:

```
Bark
```

This is called **upcasting**.

The reference type is:

```
Animal
```

but the actual object is:

```
Dog
```

Java chooses the overridden method based on the actual object at runtime.

This is called:

> **Runtime Method Dispatch** / **Dynamic Method Dispatch**

24. Upcasting

Converting a child reference into a parent reference is called **upcasting**.

```
Dog d = new Dog();

Animal a = d;
```

or:

```
Animal a = new Dog();
```

This is automatically allowed.

Why?

Because:

```
Dog IS-A Animal
```

25. Downcasting

Converting a parent reference back into a child reference is called **downcasting**.

```
Animal a = new Dog();

Dog d = (Dog) a;
```

Now:

```
d.bark();
```

can be called.

But incorrect downcasting can cause:

```
ClassCastException
```

Example:

```
Animal a = new Cat();
```

```
Dog d = (Dog) a;
```

This fails at runtime because the actual object is a `Cat`.

26. `instanceof`

We can check the actual type before downcasting.

```
Animal a = new Dog();

if (a instanceof Dog) {
  Dog d = (Dog) a;
  d.bark();
}
```

This prevents an invalid cast.

27. Inheritance vs Composition

Inheritance represents:

> **IS-A**

Example:

```
Dog IS-A Animal
```

Composition represents:

> **HAS-A**

Example:

```
Car HAS-A Engine
```

Composition:

```
class Engine {

  void start() {
    System.out.println("Engine starts");
  }
}

class Car {

  Engine engine = new Engine();
```

```
}
```

A `Car` is not an `Engine`.

Therefore, composition is more appropriate.

28. Important Example

Let's combine several concepts.

```
class Employee {

  protected String name;
  protected double salary;

  Employee(String name, double salary) {
    this.name = name;
    this.salary = salary;
  }

  void work() {
    System.out.println(name + " is working");
  }

  void display() {
    System.out.println(name);
    System.out.println(salary);
  }
}
```

Child:

```
class Developer extends Employee {

  String language;

  Developer(String name, double salary, String language) {
    super(name, salary);
    this.language = language;
  }

  @Override
  void work() {
    System.out.println(name + " is writing " + language + " code");
  }
```

```
  void code() {
    System.out.println("Developer is coding");
  }
}
```

Usage:

```
Developer d =
  new Developer("Akshat", 60000, "Java");

d.work();
d.display();
d.code();
```

Output:

```
Akshat is writing Java code
Akshat
60000.0
Developer is coding
```

Here we used:

- Inheritance
- `extends`
- `super()`
- Constructor
- `protected`
- Method overriding
- `@Override`
- Code reuse

29. Important Exam Points

Remember these points carefully:

Inheritance

```
Inheritance = acquiring properties and behaviors of another class
```

Keyword

```
extends
```

Parent class

Also called:

```
Superclass
Base class
```

Child class

Also called:

```
Subclass
Derived class
```

Multiple inheritance through classes

Not supported in Java.

Multiple inheritance through interfaces

Supported.

Constructors

Not inherited.

Private members

Cannot be directly accessed by child classes.

Final class

Cannot be inherited.

Final method

Cannot be overridden.

`super`

Used to access the immediate parent class's:

```
variable
method
constructor
```

`@Override`

Used when overriding a parent method.

Object

Every Java class ultimately inherits from:

```
Object
```

30. Inheritance Hierarchy — Quick Revision

```
          Object
            |
          Animal
          /   \
        Dog    Cat
        |
      Puppy
```

Here:

```
Dog extends Animal
Cat extends Animal
Puppy extends Dog
```

Therefore:

```
Puppy → Dog → Animal → Object
```

A `Puppy` object can access accessible methods inherited through the chain.

31. One-Line Definitions for Exams

Inheritance:
Inheritance is an OOP mechanism through which one class acquires the properties and behaviors of another class.
Superclass:
The class whose properties and methods are inherited is called the superclass or parent class.
Subclass:
The class that inherits from another class is called the subclass or child class.
Method Overriding:
When a subclass provides its own implementation of an inherited method with the same

signature, it is called method overriding.

Upcasting:

Converting a child-class reference into a parent-class reference is called upcasting.

Downcasting:

Converting a parent-class reference into a child-class reference is called downcasting.

IS-A relationship:

Inheritance represents an IS-A relationship between classes.

HAS-A relationship:

Composition represents a HAS-A relationship between classes.

Quick Memory Map

```
       INHERITANCE
         |
    -------------------------
    |          |
   extends       interfaces
    |          |
  class → class    interface → class
    |
 ------------------
 |   |    |
Single Multi- Hierarchical
   level
```

And remember:

```
Inheritance
  ↓
Code Reusability
  ↓
extends
  ↓
Parent → Child
  ↓
IS-A relationship
  ↓
Method Overriding
  ↓
Runtime Polymorphism
```

- This is the core chain you should understand before moving to **polymorphism, abstract classes, and interfaces**.